



I. INTRODUÇÃO A ENGENHARIA DE SOFTWARE

I.1. PROBLEMAS COMUNS NO DESENVOLVIMENTO DE SOFTWARE

DISCIPLINA: ENGENHARIA DE SOFTWARE I

CURSO: BACHAREL EM SISTEMAS DE INFORMAÇÃO

PROFESSOR: M.E JULLIAN HERMANN CREUTZBERG

2026

SOFTWARE

- Vários paradigmas foram utilizados ao longo do tempo para tentar controlar a construção de software.
- O primeiro, o mais famoso, o mais usado e mais desastroso modo de controlar a criação de software é conhecido modelo waterfall (cascata).
- Este modelo, baseado em como carros, prédios e pontes são construídos, é o mais danoso ao processo de construir software, porque ele é baseado em conceitos de sequência e dependência.
- Assim, tem-se que uma operação só pode ser feita se operação anterior necessária a esta foi completada.
- Isso faz sentido quando se constrói um prédio, já que ninguém quer morar num edifício em que as fundações não foram bem feitas, por exemplo.
- Mas na construção de software essa dependência e sequência não existem e o dano advém justamente de querer incluí-las.

SOFTWARE

- **O que é?**
- Software de computador é o produto que profissionais de software desenvolvem e ao qual dão suporte por muitos anos.
- Esses artefatos incluem programas executáveis em computador de qualquer porte ou arquitetura.



SOFTWARE

- Do ponto de vista do engenheiro de software, o produto do trabalho são os programas, conteúdo (dados) e documentos que compõem um software de computador;
- Mas, do ponto de vista do usuário, o produto do trabalho é a informação resultante que, de algum modo, torna melhor o mundo do usuário.

```

    operation = "SELECT"
    mirror_mod.use.x = false
    mirror_mod.use.y = true
    mirror_mod.use.z = false
    elif operation == "INSERT":
        mirror_mod.use.x = false
        mirror_mod.use.y = true
        mirror_mod.use.z = false

    mirror_obj.select = 1
    modifier_obj.select = 1
    bpy.context.scene.objects.active = modifier_obj
    print("Selected" + str(modifier_obj)) # modifier obj is the active obj
    mirror_obj.select = 0
    modifier_obj.select = 0
    bpy.context.scene.objects.active = mirror_obj

```

Mas afinal...

O que é
**Engenharia
de Software?**



O QUE É ENGENHARIA DE SOFTWARE?

- A **engenharia de software** abrange um processo, um conjunto de métodos (práticas) e uma série de ferramentas que possibilitam aos profissionais desenvolverem software de altíssima qualidade.

QUEM REALIZA A ENGENHARIA DE SOFTWARE?

- Os **engenheiros de software** criam e dão suporte a ele, e praticamente todos que têm contato com o mundo industrializado o utilizam.
- Os engenheiros de software aplicam o processo de engenharia de software.

POR QUE É IMPORTANTE?

- A engenharia de software é importante porque nos capacita para o desenvolvimento de sistemas complexos dentro do prazo e com alta qualidade.
- Ela impõe disciplina a um trabalho que pode se tornar caótico, mas também permite que as pessoas produzam software de computador adaptado à sua abordagem, da maneira mais conveniente às suas necessidades.

QUAIS SÃO AS ETAPAS ENVOLVIDAS?

- Cria-se software para computadores da mesma forma que qualquer produto bem-sucedido:
 - aplicando-se um processo adaptável e ágil que conduza a um resultado de alta qualidade, atendendo às necessidades daqueles que usarão o produto.

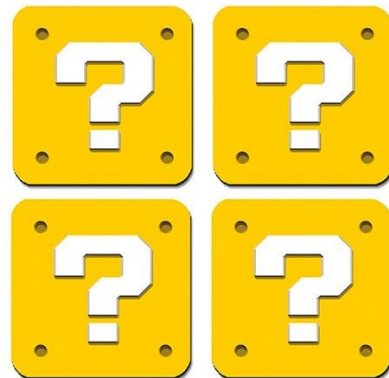
QUAL É O ARTEFATO?

- Do ponto de vista de um engenheiro de software, artefato é um conjunto de programas, conteúdo (dados) e outros artefatos que apoiam o software de computador.
- Porém, do ponto de vista do usuário, o artefato é uma ferramenta ou um produto que, de alguma forma, torna a vida dele melhor.

ALICERCES DA ENGENHARIA DE SOFTWARE

- **Métodos:**
 - Proporcionam os detalhes do “como fazer” para construir o software.
 - São os roteiros para o desenvolvimento de software.
- **Ferramentas:**
 - Proporcionam apoio automatizado ou semiautomatizado aos métodos (exemplo: ferramentas CASE).
 - São os instrumentos que proporcionam os detalhes do “como fazer”.
- **Procedimentos:**
 - Trata-se da ligação entre os anteriores.

Por que
estudamos
**Engenharia
de Software?**



PROBLEMAS COMUNS

- O desenvolvimento de um software sempre **começa com um grande problema**, que deverá ser quebrado em pequenos pedaços até que o problema seja compreendido e resolvido de forma computacional por completo.



PROBLEMAS COMUNS

- Software é diferente de um hardware:
 - O Software não é **palpável**;
 - O Software não se **desgasta** (ele fica defasado);
 - Aplicação de princípios de qualidade em software é **muito complexa**;
 - Nem sempre quem está solicitando um software **sabe realmente o que deseja**;
 - Nem sempre a **melhor** forma de construir um software é a mais **viável**.

PROBLEMAS COMUNS

1. Os **projetos nem sempre são constituídos de processo definido**, e esses são problemáticos quando tenta-se aplicar algum método;
2. Os **requisitos são mal levantados**, obtêm-se muitos requisitos implícitos;
3. Nas entrevistas realizadas com usuários **não se tem um objetivo claramente definido**, e nem usuário nem analista sabem efetivamente onde vão chegar;

PROBLEMAS COMUNS

4. O cliente **não sabe exatamente o que quer**;
5. Os **usuários têm opiniões e pontos de vista diferentes** de outros usuários sobre um mesmo procedimento operacional da empresa;
6. Quem faz não usa, ou seja, **quem constrói o software não o utiliza efetivamente**, ocasionando dificuldade na sua operação contínua;

PROBLEMAS COMUNS

7. Um **objetivo/escopo claro nem sempre é definido**, ocasionando um resultado desastroso ao final do desenvolvimento do software;
8. Os projetos têm tempo definido para início e término, **especificar um tempo coerente para finalizar o projeto é um problema**;
9. Os **custos dos projetos**, em alguns projetos são **mal dimensionados**, conflitando também com a questão do escopo do projeto, que quando mal especificado pode prejudicar tanto no tempo como no custo;

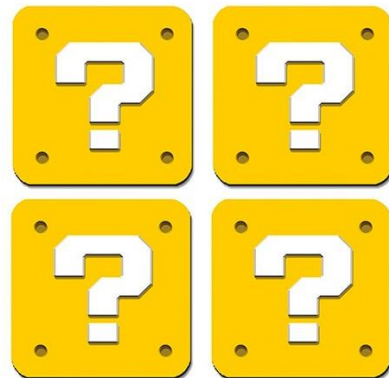
PROBLEMAS COMUNS

10. Um **método eficiente de desenvolvimento é difícil de ser implantando em uma empresa (de software) que já está em funcionamento**, existem culturas para serem quebradas, os custos por vezes são considerados altos, não existe respaldo da alta direção, sem uma ferramenta para controle eficiente um processo pode não ser seguido por todos.

ESSÊNCIA DA PRÁTICA

- A essência da solução de problemas e, conseqüentemente, a **essência da prática da engenharia de software** é:
- **Compreender o problema** (comunicação e análise).
- **Planejar uma solução** (modelagem e projeto de software).
- **Executar o plano** (geração de código).
- **Examinar o resultado para ter precisão** (testes e garantia da qualidade).

Dúvidas?



RESUMO

- A **engenharia de software** engloba processos, métodos e ferramentas que possibilitam a construção de sistemas complexos baseados em computador dentro do prazo e com qualidade.
- O processo de software incorpora cinco atividades estruturais: **comunicação, planejamento, modelagem, construção e entrega**, e elas se aplicam a todos os projetos de software.
- A prática da engenharia de software é uma atividade de **resolução de problemas** que segue um conjunto de princípios básicos.
- À medida que você aprende mais sobre engenharia de software, você começará a entender por que esses princípios devem ser considerados quando iniciamos qualquer projeto de software.

ATIVIDADE 01

- Construção de **Conceitos de Engenharia de Software**
 - Cada um receberá 1 (um) tema
 - Buscar SOMENTE nos Livros das Referências da Disciplina
 - Realizar Leitura do texto e Construir um Resumo/Conceito (20 a 30 linhas)
 - Tempo estimado: 30 minutos
 - Apresentar o conceito para a turma: 5 minutos cada
 - Adicionar o Conceito/Resumo sobre o tema no Classroom
- 1,0 ponto na nota de Atividades do Semestre

ATIVIDADE 01

- Temas:
 - 1. Métodos Ágeis
 - 2. Verificação e Validação
 - 3. Métricas em Software
 - 4. Pontos de Função
 - 5. UML
 - 6. Processo de Desenvolvimento de Software
 - 7. Padrões de Projeto
 - 8. Concepção de Análise
 - 9. Engenharia de Requisitos
 - 10. Teste de Software
 - 11. Gerencia de Configuração
 - 12. Regras de Negócio
 - 13. Requisitos Funcionais
 - 14. Requisitos Não Funcionais
 - 15. Domínio do Sistema



jullian@unidavi.edu.br



@jullianhc